SOLIDITY INTERMEDIATE

BASIC DATA TYPE

- Integer
- Bool
- Address
- Byte

Code Eater

REFERENCE DATA TYPE

Code Eater

- Strings
- Arrays
- Mappings
- Struct

REFERENCE DATA TYPE

Code Eater

- Data types such as Array, Structs, Mapping are known as reference data type.
- If we use a reference type, we always have to explicitly provide the data area where the type is stored.
- Every reference type has an additional annotation, the “data location”, about where it is stored. There are three data locations: memory, storage and calldata.

DATA LOCATIONS

Code Eater

Data Location	About
Memory	lifetime is limited to an external function call
Storage	the location where the state variables are stored, where the lifetime is limited to the lifetime of a contract
Calldata	special data location that contains the function arguments

ARRAYS

Code Eater

Fixed Array

Dynamic Array

FIXED ARRAY

Code Eater

- uint[number_of_elements] public arr;
- uint[5] public arr;

0	0	0	0	0
0	1	2	3	4

- uint[5] public arr = [10,20,30,40,50];

10	20	30	40	50
0	1	2	3	4

FIXED ARRAY

```
contract FixedSizeArray {
    // Declare a fixed-size array of uint with length 5
    uint[5] public fixedArray;

    // Function to get the length of the fixed-size array
    function getLength() public view returns (uint) {    359 gas
        return fixedArray.length;
    }

    // Function to update an element in the fixed-size array
    function updateElement(uint index, uint value) public {    inf
        require(index < fixedArray.length, "Index out of bounds");
        fixedArray[index] = value;
    }
}
```

Click Here

FIXED ARRAY EXAMPLE 2

Click Here

DYNAMIC ARRAY

Code Eater

- uint[] public arr;
- To insert an element we use arr.push(element)
- To remove an element we use arr.pop() . It removes the last element every time it is executed.
- To find the length of an arry we use arr.length. It returns the length of the array in uint data type.

DYNAMIC ARRAY

```
contract DynamicSizeArray {
    // Declare a dynamic-size array of uint
    uint[] public dynamicArray;

    // Function to add an element to the dynamic-size array
    function addElement(uint value) public {    46873 gas
        dynamicArray.push(value);
    }

    // Function to get the length of the dynamic-size array
    function getLength() public view returns (uint) {    2467 gas
        return dynamicArray.length;
    }

    // Function to remove the last element from the dynamic-size array
    function removeLastElement() public {    31554 gas
        require(dynamicArray.length > 0, "Array is already empty");
        dynamicArray.pop();
    }
}
```

Click Here

STRING DATA TYPE

```
contract StringExample {
    // State variable to store a string
    string public myString;

    // Function to set a new string
    function setString(string memory newString) public {
        myString = newString;
    }

    // Function to get the current string (optional, since we h
    function getString() public view returns (string memory) {
        return myString;
    }
}
```

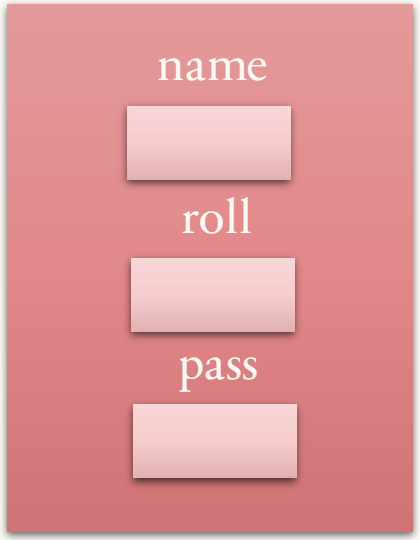
Click Here

STRUCT

Code Eater

- Struct is a complex data type. A complex data type is **usually a composite of other existing data types.**

```
struct Student{
    string name;
    uint roll;
    bool pass;
}
```



- struct_type public var_name;
- Student public s1;

STRUCT

```
contract Classroom {
    struct Student {
        string name;
        uint roll;
        bool pass;
    }

    Student public s1;

    function insert() public returns (Student memory) {
        s1.name = "Kshitij";
        s1.roll = 12;
        s1.pass = true;
        return s1;
    }
}
```

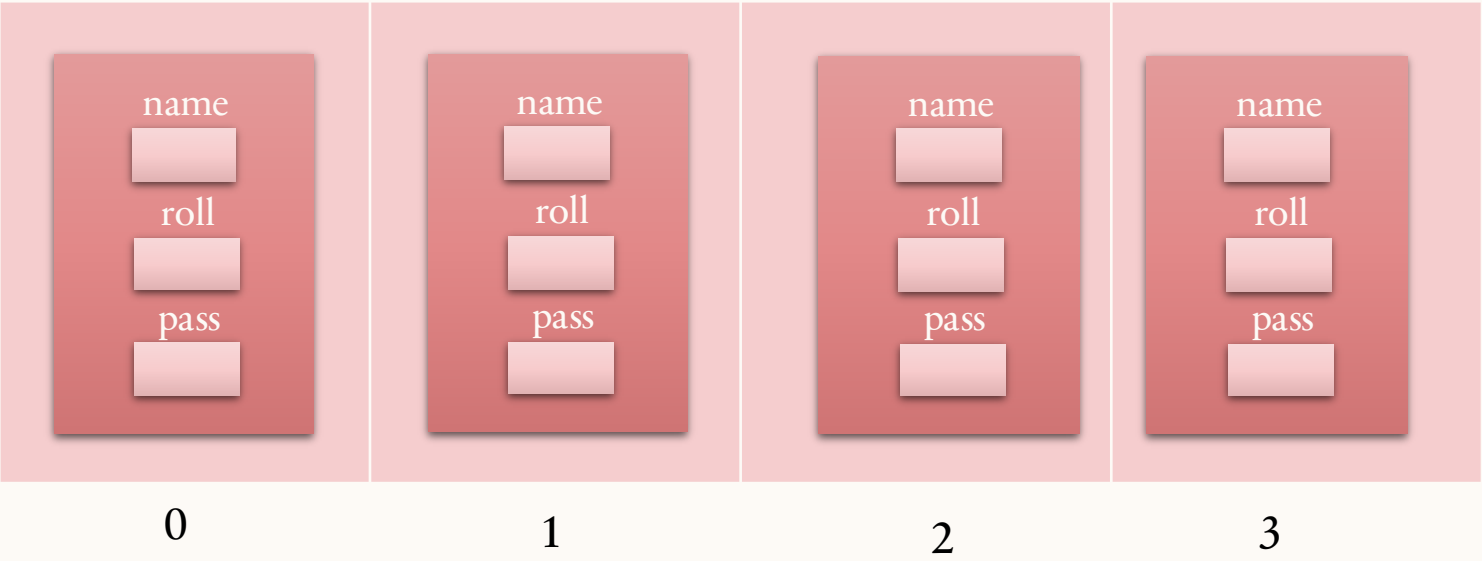
Click Here

ARRAY OF STRUCT

Code Eater

```
struct Student{
    string name;
    uint roll;
    bool pass;
}
```

Student[4] public s1;



MAPPING

Code Eater

- Concept of keys and values.
- mapping(key => value)
- mapping(uint => string) public data;

```
contract Grades {
    // Define a mapping from student addresses to their grades
    mapping(address => uint) public grades;

    // Function to set the grade for a student
    function setGrade(address student, uint grade) public {    2283
        grades[student] = grade;
    }

    // Function to get the grade of a student
    function getGrade(address student) public view returns (uint) {
        return grades[student];
    }
}
```

Click Here

SIMPLE MAPPING

Code Eater

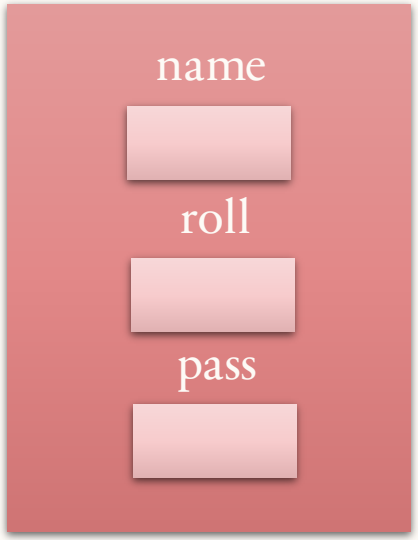
- mapping(uint => string) public data;

_roll		name
1	→	Ravi
5	→	John
40	→	Alice
50	→	Akash

MAPPING WITH STRUCT

Code Eater

- mapping(uint=>Student) public data;

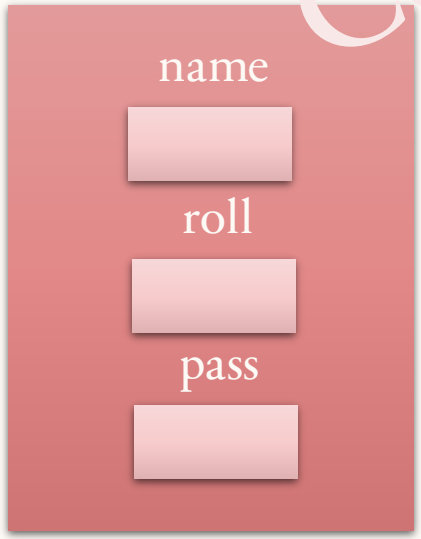


Student

- mapping(uint=>Student) public data;

uint	Student
0	name roll pass
12	name roll pass
100	name roll pass

```
voterDetails[_voterId].voterAddress==msg.sender
```



Student

Code Eater

MAPPING WITH STRUCT

Code Eater

roll	_name	_class	_age
1	John	7	13
5	Alice	8	14
10	Ravi	3	9

MAPPING WITH ARRAY

Code Eater

mapping(address=>uint[ ]) private marks;

address	uint[ ]
0xabc	10 20 30 40 50
0xdef	1 2 3
0x121	121 221 243 52
0x23f	12 22 23
0x73e	10 20 30 40 50

NESTED MAPPING

Code Eater

mapping(address=>mapping(address=>bool)) private check;

	address1	address2	address3
address1		true	
address2			false
address3	false		

check[address1][address2] = true

check[address2][address3] = false

check[address3][address1] = false

NESTED MAPPING

Code Eater

```
mapping(uint=>mapping(uint=>bool)) private check;
```

	0	1	2
0		true	
1			false
2	false		

check[0][1] = true

check[1][2] = false

check[2][0] = false

NESTED MAPPING

Code Eater

mapping(address=>mapping(address=>bool)) ownership;

	address1	address2	address3
address1		true	
address2			false
address3	false		

ownership[address1][address2] = true

ownership[address2][address3] = false

ownership[address3][address1] = false

```
// contracts/GLDToken.sol
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

// storage-Characteristics:
// Persistent: Data is stored on the blockchain and persists between
function calls and transactions.
// Expensive: Operations on storage are costly in terms of gas because it
involves writing to the blockchain.
// Default Location: State variables are stored in storage by default.

// memory-Characteristics:
// Temporary: Data is not saved on the blockchain and exists only while
the function is executing.
// Cheaper: Operations on memory are less expensive compared to storage.
// Explicitly Specified: You need to explicitly specify memory for dynamic
arrays and structs.

// 3. calldata-Definition
// -calldata is a read-only data location used for function arguments. It
is used when dealing with external function calls and is immutable (cannot
be modified).
// Characteristics:
// Read-Only: Data in calldata cannot be modified. It is used for function
parameters in external function calls.
// Efficient: Accessing calldata is more gas-efficient for function
arguments compared to copying them into memory.
// Default for External Functions: Function parameters of external
functions use calldata by default.
```

FIXEDSIZED-ARRAY(10.14)

```
// SPDX-License-Identifier: GPL-3.0

pragma solidity >=0.8.2 <0.9.0;

contract FixedArrayExample {
```

```
    // Define a fixed-size array with 5 elements
    uint256[5] public fixedArray;

    // Function to set the values of the fixed-size array
    function setArray(
        uint256 _array1,
        uint256 _array2,
        uint256 _array3,
        uint256 _array4,
        uint256 _array5
    ) public {
        fixedArray[0] = _array1;
        fixedArray[1] = _array2;

        fixedArray[2] = _array3;
        fixedArray[3] = _array4;

        fixedArray[4] = _array5;
    }

    // Function to get a specific element from the fixed-size array
    function getElement(uint256 index) public view returns (uint256) {
        require(index < 5, "Index out of bounds"); // Check index bounds
        return fixedArray[index];
    }

    // Function to get the entire fixed-size array
    function getArray() public view returns (uint256[5] memory) {
        return fixedArray;
    }

    // Function to update a specific element in the fixed-size array
    function updateElement(uint256 index, uint256 value) public {
        require(index < 5, "Index out of bounds"); // Check index bounds
        fixedArray[index] = value;
    }
}
```

DYNAMIC-ARRAY(10.15)

```
// SPDX-License-Identifier: GPL-3.0

pragma solidity >=0.8.2 <0.9.0;

contract DynamicArrayExample {
    // Define a dynamic array of uint256
    uint256[] public dynamicArray;

    // Function to add an element to the end of the dynamic array
    function addElement(uint256 _element) public {
        dynamicArray.push(_element);
    }

    // Function to get the length of the dynamic array
    function getArrayLength() public view returns (uint256) {
        return dynamicArray.length;
    }

    // Function to get a specific element from the dynamic array
    function getElement(uint256 _index) public view returns (uint256) {
        require(_index < dynamicArray.length, "Index out of bounds");
        return dynamicArray[_index];
    }

    // Function to remove the last element from the dynamic array
    function removeLastElement() public {
        require(dynamicArray.length > 0, "Array is empty");
        dynamicArray.pop();
    }

    // Function to update an element at a specific index in the dynamic
array
    function updateElement(uint256 _index, uint256 _value) public {
        require(_index < dynamicArray.length, "Index out of bounds");
        dynamicArray[_index] = _value;
    }

    // Function to remove an element at a specific index (by shifting)
    function removeElementAt(uint256 _index) public {
```

```
        require(_index < dynamicArray.length, "Index out of bounds");
        for (uint256 i = _index; i < dynamicArray.length - 1; i++) {
            dynamicArray[i] = dynamicArray[i + 1];
        }
        dynamicArray.pop(); // Remove the last duplicate element
    }

    // Function to get the entire dynamic array
    function getArray() public view returns (uint256[] memory) {
        return dynamicArray;
    }
}
```

STRING(10.16)

```
// SPDX-License-Identifier: GPL-3.0

pragma solidity >=0.8.2 <0.9.0;

contract StringExample {
    // State variable to store a string
    string public myString;

    // Function to set a new string
    function setString(string memory newString) public {
        myString = newString;
    }

    // Function to get the current string (optional, since we have a
public variable)
    function getString() public view returns (string memory) {
        return myString;
    }
}
```

STRUCT(10.17)

```
// SPDX-License-Identifier: GPL-3.0
```

```
pragma solidity >=0.8.2 <0.9.0;

contract Classroom {
    struct Student {
        string name;
        uint256 roll;
        bool pass;
    }

    Student public s1;

    function insert() public returns (Student memory) {
        s1.name = "dhruv";
        s1.roll = 12;
        s1.pass = true;
        return s1;
    }
}
```

STRUCT_WITH_MAPPING(10.18)

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract StudentRecords {
    // Define a struct to represent a student
    struct Student {
        uint256 id;
        string name;
        uint256 age;
    }

    // Mapping to store student ID to Student struct
    mapping(uint256 => Student) private students;
         //    _id    onestruct        collections

    // Function to add or update a student
    function addOrUpdateStudent(
        uint256 _id,
        string memory _name,
```

```
        uint256 _age
    ) public {
        students[_id] = Student(_id, _name, _age);
    }

    // Function to get a student by ID
    function getStudent(uint256 _id)
        public
        view
        returns (
            uint256,
            string memory,
            uint256
        )
    {
        require(bytes(students[_id].name).length != 0, "Student not
found");
        Student memory student = students[_id];
        return (student.id, student.name, student.age);
    }

    // Function to delete a student by ID
    function deleteStudent(uint256 _id) public {
        require(bytes(students[_id].name).length != 0, "Student not
found");
        delete students[_id];
    }
}
```

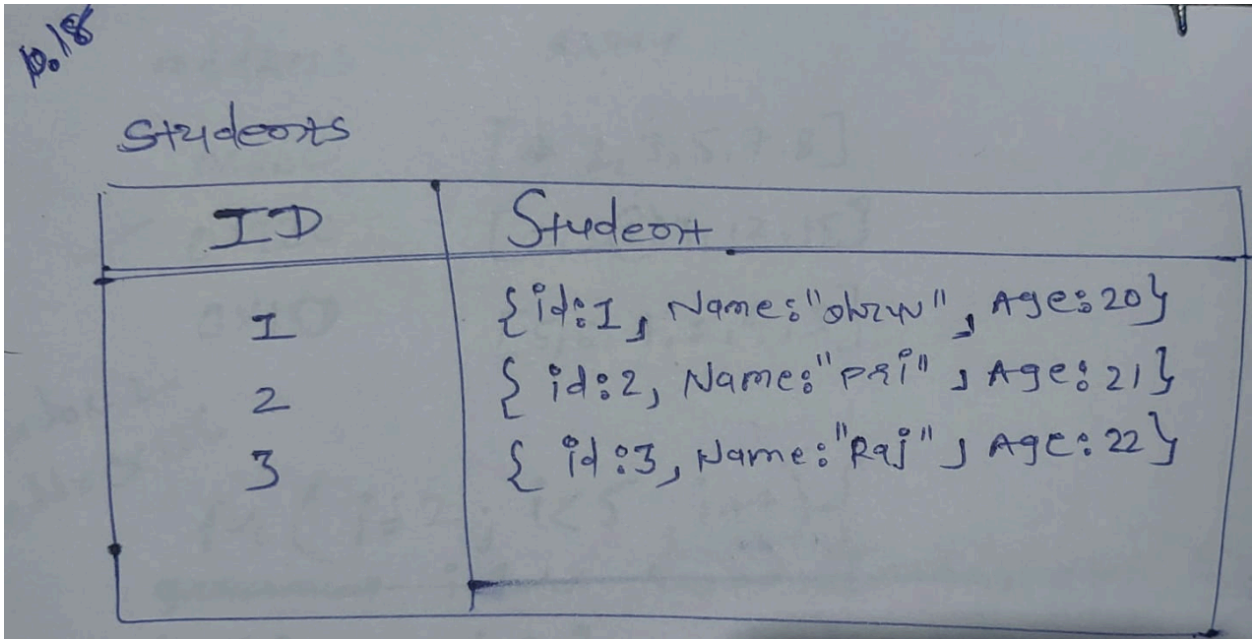
ARRAY_OF_STRUCT(10.19)

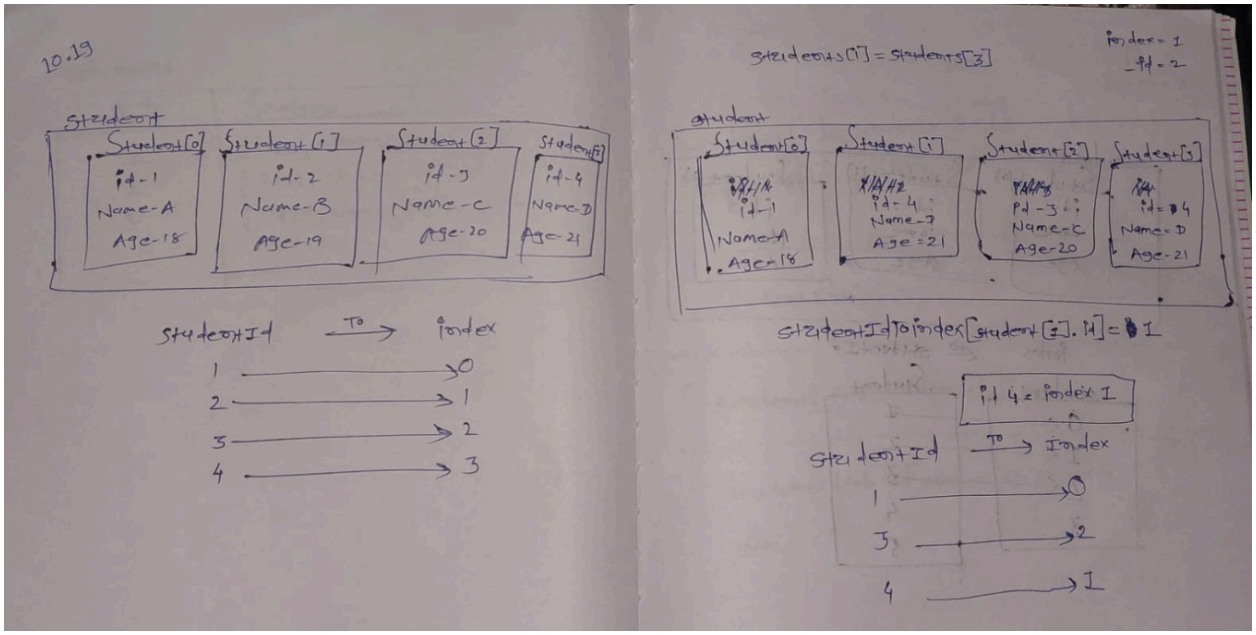
```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;
//index is main here
contract StudentManagement {
    // Define a struct to represent a student
    struct Student {
        uint256 id;
        string name;
```

```
        uint256 age;
    }

    // Array to store students
    Student[] public students;

    // Mapping to store student ID to array index
    mapping(uint256 => uint256) public studentIdToIndex;
    //     studentid  arrayindex        collection

    // Function to add a new student
    function addStudent(
        uint256 _id,
        string memory _name,
        uint256 _age
    ) public {
        // Create a new student struct and add it to the array
        students.push(Student(_id, _name, _age));
        // Update the mapping with the new student's index
        studentIdToIndex[_id] = students.length - 1;
    }

    // Function to get a student by index
    function getStudentByIndex(uint256 _index)
        public
        view
        returns (
            uint256,
            string memory,
            uint256
        )
    {
        require(_index < students.length, "Index out of bounds");
        Student memory student = students[_index];
        return (student.id, student.name, student.age);
    }

    // Function to get a student by ID
    function getStudentById(uint256 _id)
        public
```

```
        view
        returns (
            uint256,
            string memory,
            uint256
        )
    {
        uint256 index = studentIdToIndex[_id];
        require(index < students.length, "Student not found");
        Student memory student = students[index];
        return (student.id, student.name, student.age);
    }

    // Function to update a student's details by ID
    function updateStudent(
        uint256 _id,
        string memory _name,
        uint256 _age
    ) public {
        uint256 index = studentIdToIndex[_id];
        require(index < students.length, "Student not found");
        Student storage student = students[index];
        student.name = _name;
        student.age = _age;
    }

    // Function to remove a student by ID
    function removeStudent(uint256 _id) public {
        uint256 index = studentIdToIndex[_id];
        require(index < students.length, "Student not found");

        // Move the last student to the place of the one to be removed
        students[index] = students[students.length - 1];
        // Update the index mapping
        studentIdToIndex[students[index].id] = index;

        // Remove the last student
        students.pop();
        // Remove the index mapping
        delete studentIdToIndex[_id];
```

```
    }
}
```





ARRAY_OF_STRUCT(10.20)

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract Library {
```

```
// Define a struct to represent a book
struct Book {
    uint256 id;
    string title;
    string author;
}

// Array to store books
Book[] public books;

// Mapping to store book ID to array index
mapping(uint256 => uint256) public bookIdToIndex;

// Function to add a new book
function addBook(
    uint256 _id,
    string memory _title,
    string memory _author
) public {
    // Create a new book struct and add it to the array
    books.push(Book(_id, _title, _author));
    // Update the mapping with the new book's index
    bookIdToIndex[_id] = books.length - 1;
}

// Function to get a book by index
function getBookByIndex(uint256 _index)
    public
    view
    returns (
        uint256,
        string memory,
        string memory
    )
{
    require(_index < books.length, "Index out of bounds");
    Book memory book = books[_index];
    return (book.id, book.title, book.author);
}
```

```
    // Function to get a book by ID
    function getBookById(uint256 _id)
        public
        view
        returns (
            uint256,
            string memory,
            string memory
        )
    {
        uint256 index = bookIdToIndex[_id];
        require(index < books.length, "Book not found");
        Book memory book = books[index];
        return (book.id, book.title, book.author);
    }
}
```

MAPPING(10.21)

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract NameToAddressMapping {
    // Mapping from string (name) to address
    mapping(string => address) private nameToAddress;

    // Function to set an address for a given name
    function setAddress(string memory _name, address _address) public {
        nameToAddress[_name] = _address;
    }

    // Function to get the address associated with a given name
    function getAddress(string memory _name) public view returns (address)
{
        require(nameToAddress[_name] != address(0), "Address not found");
        return nameToAddress[_name];
    }

    // Function to delete the address associated with a given name
    function deleteAddress(string memory _name) public {
```

```
        require(nameToAddress[_name] != address(0), "Address not found");
        delete nameToAddress[_name];
    }
}
```

MAPPIN_WITH_ARRAY(10.22)

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract AddressToArrayMapping {
    // Mapping from an address to an array of uint256 values
    mapping(address => uint256[]) private addressToValues;

    // Function to add a value to the array for a given address
    function addValue(address _address, uint256 _value) public {
        addressToValues[_address].push(_value);
    }

    // Function to get the array of values for a given address
    function getValues(address _address)
        public
        view
        returns (uint256[] memory)
    {
        return addressToValues[_address];
    }

    // Function to remove a value at a specific index for a given address
    function removeValueAt(address _address, uint256 _index) public {
        require(
            _index < addressToValues[_address].length,
            "Index out of bounds"
        );

        // Shift elements to the left to overwrite the value at _index
        for (
            uint256 i = _index;
```

```
            i < addressToValues[_address].length - 1;
            i++
        ) {
            addressToValues[_address][i] = addressToValues[_address][i + 1];
        }

        // Remove the last element
        addressToValues[_address].pop();
    }
}
```

address	Array
0X9bc312F2	[1, 2, 3, 4, 5, 6]
0X12ABCd23	[2, 4, 9, 8, 7, 3] (i=2)
0X31A9bcde	[1, 2, 4, 5, 6, 3]

for (i=2 ; i < 5 , i++) {

loop = 2	2 = 3	[2, 4, 8, 8, 7, 3]
loop - 3	3 = 4	[2, 4, 8, 7, 7, 3]
loop - 4	4 = 5	[2, 4, 8, 7, 3, 3] ↑ Remove

}

CONSTANT(10.23)

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract ConstantsExample {
    address public constant add1 =
0x5B38Da6a701c568545dCfcB03FcB875f56beddC4; //351 gas
    address public add2 = 0x5B38Da6a701c568545dCfcB03FcB875f56beddC4;
//2505 gas
}
```

ENUM(10.24)

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract OrderStatus {
    // Define an enum to represent the status of an order
    enum Status {
        Pending,
        Shipped,
        Accepted,
        Rejected,
        Cancelled
    }

    // Variable to store the current status of an order
    Status public currentStatus;

    // Function to set the status of the order
    function setStatus(Status _status) public {
        currentStatus = _status;
    }

    // Function to get the current status of the order as a string
    function getStatus() public view returns (string memory) {
        if (currentStatus == Status.Pending) return "Pending";
        if (currentStatus == Status.Shipped) return "Shipped";
        if (currentStatus == Status.Accepted) return "Accepted";
```

```
        if (currentStatus == Status.Rejected) return "Rejected";
        if (currentStatus == Status.Cancelled) return "Cancelled";
        return "Unknown";
    }
}
```